Lovely Professional University

School of Computer Science and Engineering

UpGrad Campus, Phagwara, Punjab, India

SYNOPSIS

for the B.Tech Final Year Project

Flower Species Recognition Using CNN

Submitted by:

Utkarsh Patel

Reg no. 12307357

Course Code: CSM422

Session: July' 26 – Dec '26

ABSTRACT

Identifying a flower species from a photograph is a small but representative image-classification problem. This project builds and evaluates Convolutional Neural Networks (CNNs) that classify flower photographs into five species: daisy, dandelion, roses, sunflowers and tulips. The TensorFlow flower photos dataset of 3,670 images was loaded with Keras, resized to 128 × 128 pixels and divided 80:20 into 2,936 training and 734 validation images. Two models were compared: a baseline CNN with three convolution–pooling blocks, and an improved CNN that adds random flip, rotation and zoom augmentation together with dropout layers. The baseline reached 99.18% training accuracy but only 66.89% final validation accuracy, a clear case of overfitting. The improved model reached 80.45% training accuracy and 75.34% final validation accuracy (best 76.02%), with a much smaller gap between the two. It was therefore selected as the final model. Its classification report gives a macro-average F1-score of 0.74, with 181 of 734 validation images misclassified; roses and tulips were the most frequently confused classes. All reported figures are validation results, since no separate test set was used. The report concludes that regularization improved generalization, while moderate accuracy and the lack of an independent test set limit how far the results can be trusted.

Keywords: image classification, convolutional neural network, data augmentation, dropout, overfitting, TensorFlow, Keras

1: Introduction

1.1 Flower Recognition and Computer Vision

Flowers are among the most recognisable subjects in photography, yet telling species apart can be hard for a non-specialist: a daisy and a dandelion are both small with yellow centres, and roses and tulips can look alike at close range. Computer vision supports plant identification by learning visual cues such as petal shape, colour, texture and layout directly from labelled photographs, instead of relying on hand-written rules.

1.2 Deep Learning and Convolutional Neural Networks

Deep learning uses networks with many layers to learn useful representations from raw data [9]. For images, the dominant architecture is the Convolutional Neural Network (CNN). Instead of connecting every pixel to every neuron, a CNN slides small filters across the image, so one pattern detector is reused at every location. Early layers respond to edges and colour patches, and deeper layers combine them into petals, centres and larger structures. CNNs were popularised for document recognition by LeCun et al. and for large-scale photograph classification by Krizhevsky et al.

1.3 Motivation and Applications

The motivation is to understand, through a small reproducible experiment, how a CNN is built and trained and how overfitting appears and can be reduced. Similar systems can support plant-identification apps, educational tools, biodiversity records and quality sorting in horticulture. This report documents only the notebook experiment, not a deployed application.

Problem Statement and Objectives

2.1 Problem Statement

Given a colour photograph of a single flower, the system must assign it to one of five species classes. This is multi-class image classification in which each image has one correct label, and the model must work well on images it did not see during training.

2.2 Objectives

Main objective: to develop a CNN-based image classification system that identifies the species of a flower from an input image. The specific objectives are to:

1. load and prepare the TensorFlow flower photos dataset with a training/validation split;
2. build and train a baseline CNN and observe whether it overfits;
3. build an improved CNN with data augmentation and dropout and compare it with the baseline;

4. evaluate the selected model with a confusion matrix, classification report and misclassified examples; and
5. demonstrate prediction on a new image and save and reload the trained model.

2.3 Scope, Expected Outcome and Challenges

The scope is limited to five flower classes, the 3,670 dataset images and a 128 × 128 input size; web, mobile or API deployment is outside it. The expected outcome is a trained CNN whose validation accuracy is well above chance, together with evidence of how regularization changes generalization. The task is challenging because photographs vary in lighting, angle, background and scale, some species look similar, one species can look very different as a bud and as a bloom, and a few hundred images per class allow a large network to memorise the training set.

Dataset Description and Exploratory Data Analysis

3.1 Source and Class Distribution

The experiment uses the TensorFlow flower photos dataset [1], downloaded in the notebook with `tf.keras.utils.get_file` from the official archive `flower_photos.tgz` and extracted automatically. Each sub-folder holds the JPEG images of one class and its name serves as the label. The five classes are daisy, dandelion, roses, sunflowers and tulips, indexed alphabetically (0 to 4). The notebook counts 3,670 images in total (Table 1, Figures 1 and 2).

Table 1: Class-wise image distribution of the dataset

Class	Images	Share of dataset
Daisy	633	17.2%
Dandelion	898	24.5%
Roses	641	17.5%
Sunflowers	699	19.0%
Tulips	799	21.8%
Total	**3,670**	**100%**

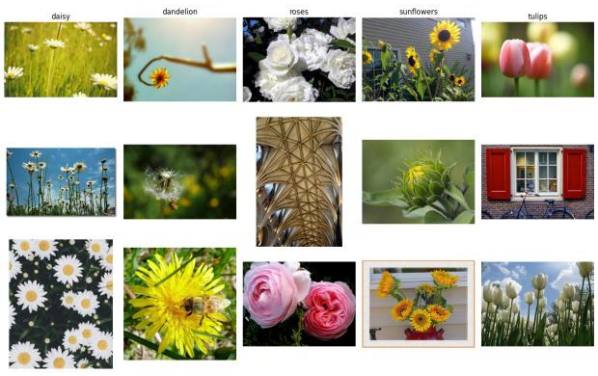


Figure 1: Random sample images (three per class), as plotted in the notebook

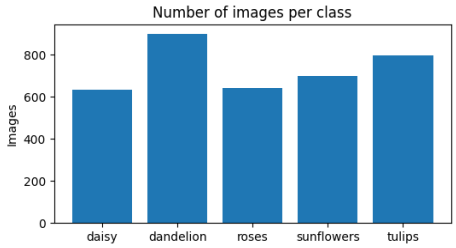


Figure 2: Number of images per class

The dataset is moderately imbalanced. Dandelion is the largest class (898 images) and daisy the smallest (633), a ratio of about 1.4 to 1. This is mild, so no resampling or class weighting was applied, but a classifier may lean slightly towards larger classes, which matters when reading per-class recall in Chapter 7.

3.2 Splitting and Preprocessing

Images were loaded with `image_dataset_from_directory` using a 20% validation split and seed 123, giving 2,936 training and 734 validation images. Each image is resized to 128 × 128 pixels and held as three RGB channels, i.e. a 128 × 128 × 3 array of values from 0 to 255, and images are served in batches of 32 (92 training batches per epoch). The pipeline caches the data, shuffles the training set (buffer of 1,000) and prefetches batches. Pixel values are scaled to 0–1 by a Rescaling layer inside the model.

Theoretical Background

4.1 From Neural Networks to CNNs

A conventional artificial neural network (ANN) flattens an image into one vector and connects it to fully connected layers. For a 128 × 128 × 3 image that means 49,152 inputs, many weights, and no memory of which pixels were neighbours. A CNN keeps the image grid and applies a convolution: a small filter (here 3 × 3) slides over the image, and at each position its weights are multiplied with the pixels beneath, summed and added to a bias. Each filter learns one pattern, and the grid of its responses is a feature map. Because filter weights are shared across positions, a CNN needs far fewer parameters and detects a pattern wherever it appears.

4.2 Layers Used in the Models

ReLU outputs max(0, x), adding the non-linearity that lets stacked layers model complex patterns. **Max pooling** keeps the largest value in each 2 × 2 window, halving height and width, which cuts computation and tolerates small shifts. **Flatten** turns the final feature maps into one vector, and a **Dense** layer combines all those features. The final dense layer has five neurons with **softmax**, which converts five raw scores into positive probabilities that sum to one; the largest gives the predicted class.

4.3 Training Concepts

Sparse categorical cross-entropy compares the predicted probabilities with the true integer label and penalises confident wrong answers heavily. **Adam** is an adaptive optimizer that scales each weight's update using running averages of past gradients. One **epoch** is a full pass over the training set, and the **batch size** (32) is the number of images processed per weight update.

4.4 Overfitting, Data Augmentation and Dropout

Overfitting means a model memorises training examples and fails on new data: training accuracy stays high while validation accuracy lags and validation loss rises. **Data augmentation** applies random flips, rotations and zooms to training images so the model sees varied versions of each photograph and learns features that do not depend on exact position. **Dropout** randomly switches off a fraction of neurons at each training step, so no neuron can be relied on alone. Both are active only during training.

4.5 Evaluation Metrics

Accuracy is the fraction of images classified correctly. For one class, **precision** is the share of images predicted as that class that truly belong to it, **recall** is the share of that class's images that were found, and **F1-score** is their harmonic mean . **Support** is the number of true images of the class. A **confusion matrix** tabulates true against predicted classes.

5: System Design and Methodology

5.1 System Pipeline

The notebook follows the pipeline in Figure 3: the dataset is loaded, resized and batched, split into training and validation subsets, two CNNs are built and trained, the better one is chosen by validation accuracy, and that model is evaluated in detail and used for a single-image prediction.

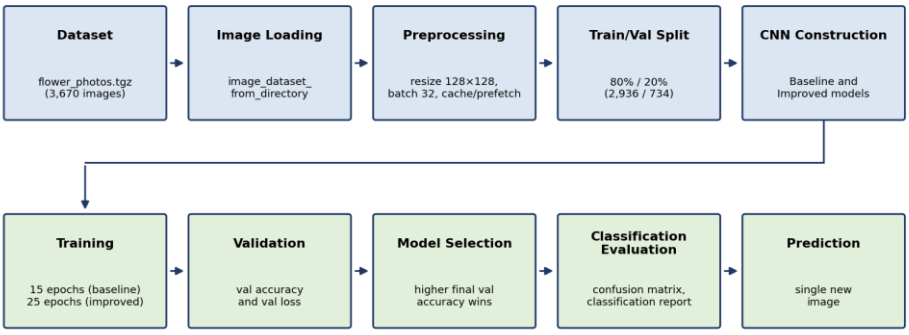


Figure 3: System pipeline of the flower recognition experiment

5.2 Baseline and Improved CNN Methodology

One function, build_cnn(improved), builds both models, so they share the same core: three Conv2D layers with 32, 64 and 128 filters (3 × 3, ReLU, "same" padding), each followed by 2 × 2 max pooling, then Flatten, a 128-neuron Dense layer and a 5-neuron softmax layer. Each model has 4,288,325 trainable parameters, almost all in the first Dense layer. Table 2 lists the differences, and Figures 4 and 5 show the notebook's layer summaries.

Table 2: Differences between the baseline and improved CNNs

Component	Baseline CNN	Improved CNN
Data augmentation	None	RandomFlip (horizontal), RandomRotation (0.1), RandomZoom (0.1)
Dropout	None	0.25 after the last pooling layer; 0.5 before the output layer
Epochs	15	25
Convolutional core	32-64-128 filters	32-64-128 filters (same)

5.3 Training, Evaluation and Model Selection

Both models use the Adam optimizer, sparse categorical cross-entropy loss and accuracy as the metric. The baseline trains for 15 epochs and the improved model for 25, since regularized models usually learn more slowly. Random seeds were fixed at 42, and the notebook ran on TensorFlow 2.21.0 with a GPU in Google Colab. After training, the model with the higher final validation accuracy becomes the final model, and its validation predictions feed the confusion matrix, classification report and error analysis.

Layer	Output shape	Params
rescaling	(128, 128, 3)	0
conv2d	(128, 128, 32)	896
max_pooling2d	(64, 64, 32)	0
conv2d_1	(64, 64, 64)	18,496
max_pooling2d_1	(32, 32, 64)	0
conv2d_2	(32, 32, 128)	73,856
max_pooling2d_2	(16, 16, 128)	0
flatten	(32768)	0
dense	(128)	4,194,432
dense_1	(5)	645
Total		**4,288,325**

Figure 4: Baseline CNN summary (batch dimension omitted)

Layer	Output shape	Params
random_flip	(128, 128, 3)	0
random_rotation	(128, 128, 3)	0
random_zoom	(128, 128, 3)	0
rescaling_1	(128, 128, 3)	0
conv2d	(128, 128, 32)	896
max_pooling2d	(64, 64, 32)	0
conv2d_1	(64, 64, 64)	18,496
max_pooling2d_1	(32, 32, 64)	0
conv2d_2	(32, 32, 128)	73,856
max_pooling2d_2	(16, 16, 128)	0
dropout	(16, 16, 128)	0
flatten	(32768)	0
dense	(128)	4,194,432
dropout_1	(128)	0
dense_1	(5)	645
Total		**4,288,325**

Figure 5: Improved CNN summary (batch dimension omitted)

Chapter 6: Implementation

The project is a single Jupyter notebook in Python, run in Google Colab. TensorFlow and Keras provide the data pipeline, layers and training loop; NumPy and Pandas handle arrays and tables; Matplotlib and Seaborn draw the plots; and Scikit-learn supplies the evaluation metrics.

6.1 Data Loading

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir, validation_split=0.2, subset="training",
    seed=123, image_size=(128, 128), batch_size=32)
# val_ds is built the same way with subset="validation"
train_ds = train_ds.cache().shuffle(1000).prefetch(AUTOTUNE)
```

6.2 Model Construction, Compilation and Training

build_cnn adds the augmentation layers and the two dropout layers only when improved=True. Each model is then compiled and trained with model.fit, and the returned History objects are plotted by a helper function.

```
model.add(layers.Conv2D(32, 3, padding="same", activation="relu"))
model.add(layers.MaxPooling2D())          # repeated for 64 and 128 filters
if improved: model.add(layers.Dropout(0.25))
model.add(layers.Flatten()); model.add(layers.Dense(128, activation="relu"))
if improved: model.add(layers.Dropout(0.5))
model.add(layers.Dense(5, activation="softmax"))
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

6.3 Evaluation, Prediction and Saving

One pass over the validation set collects true labels, predicted labels (the arg-max of the softmax output) and images in the same order, which feed the confusion matrix, the classification report and a grid of the first twelve misclassified images. predict_image loads and resizes a new file, adds a batch dimension and prints the five class probabilities. The

selected model is saved in the .keras format and reloaded, and the reloaded model reproduced the validation accuracy of 0.7534.

```
best_model.save("flower_cnn.keras")
loaded_model = tf.keras.models.load_model("flower_cnn.keras")
```

7: Results and Discussion

7.1 Baseline versus Improved CNN

Table 3 reproduces the notebook's results table. All values are training or validation metrics; there is no separate test set.

Table 3: Training and validation results of the two models

Metric	Baseline CNN	Improved CNN
Training epochs	15	25
Final training accuracy	99.18%	80.45%
Final validation accuracy	66.89%	75.34%
Best validation accuracy	66.89%	76.02%
Final training loss	0.0290	0.5012
Final validation loss	2.2104	0.6872
Training–validation accuracy gap	32.29 points	5.11 points

The baseline learns the training set almost perfectly but generalizes poorly. Training accuracy climbs from 37.8% to 99.2% (Figure 6), while validation accuracy stays between roughly 58% and 67% after the first epochs. Validation loss is lowest at epoch 4 (0.9212) and then rises to 2.2104 while training loss keeps falling (Figure 8, left). With the 32.29-point gap, this is the standard signature of overfitting: the network memorises training images instead of learning features that transfer.

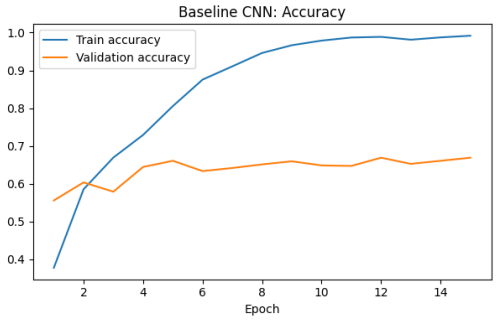


Figure 6: Baseline CNN, training vs validation accuracy

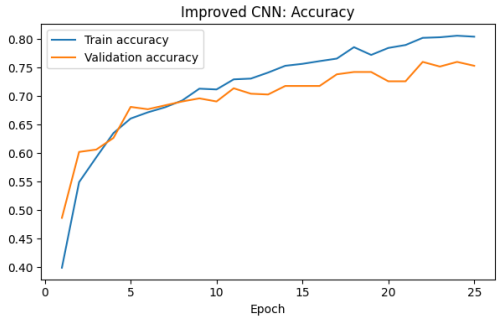


Figure 7: Improved CNN, training vs validation accuracy

The improved model behaves differently. Training accuracy rises more slowly to 80.45%, but validation accuracy reaches 75.34% at the final epoch and 76.02% at its best (Figure 7), and validation loss ends at 0.6872, close to the training loss of 0.5012 (Figure 8). The gap shrinks to 5.11 points. Two cautions apply. Keras measures training accuracy while augmentation and

dropout are active, which lowers the training figure, so the gap is not perfectly like-for-like; and validation accuracy fluctuates between epochs (72.62% at epoch 21, 76.02% at epoch 22), so one-point differences should not be over-interpreted. The results are consistent with augmentation and dropout improving generalization, but the notebook does not test each technique separately, so their individual contributions are not established.

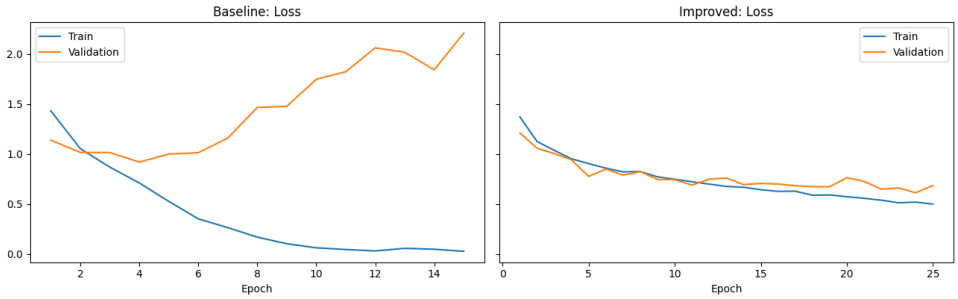


Figure 8: Training vs validation loss for the baseline (left) and improved (right) CNN

7.2 Classification Report of the Selected Model

The improved CNN was selected because its validation accuracy (75.34%) exceeds the baseline's (66.89%). Table 4 is its classification report on the 734 validation images.

Table 4: Classification report of the improved CNN (validation set)

Flower class	Precision	Recall	F1-score	Support
Daisy	0.85	0.65	0.74	129
Dandelion	0.83	0.76	0.79	176
Roses	0.67	0.57	0.61	120
Sunflowers	0.84	0.88	0.86	152
Tulips	0.63	0.85	0.73	157
Macro average	0.76	0.74	0.74	734
Weighted average	0.76	0.75	0.75	734

Sunflowers are recognised best (F1 = 0.86), followed by dandelion (0.79); roses are weakest (F1 = 0.61, recall 0.57). Tulips have high recall (0.85) but low precision (0.63): most tulips are found, but many images labelled tulip belong to other classes. High recall alone therefore does not imply a reliable class, since a model that over-predicts a class collects most of its true members along with false positives. Daisy shows the opposite pattern (precision 0.85, recall 0.65): daisy predictions are usually right, but many daisies receive another label.

7.3 Confusion Matrix and Misclassification Analysis

In the confusion matrix (Figure 9) the diagonal holds 553 correct predictions (84 + 133 + 68 + 134 + 134), so 181 of 734 images (24.7%) are misclassified. The largest single error is roses predicted as tulips: 43 of 120 rose images (about 36%). Other notable errors are 19 daisies predicted as dandelions and 14–15 dandelions predicted as sunflowers or tulips. The 43 rose-

to-tulip errors explain most of tulip's low precision, since 212 images were predicted as tulips and only 134 were correct.

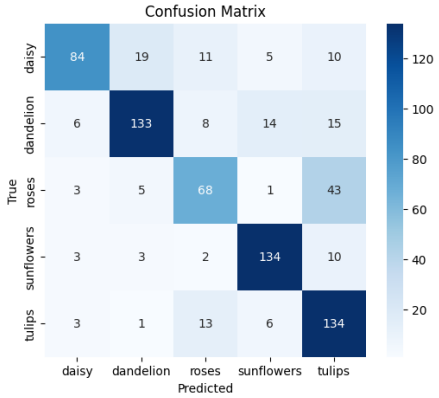


Figure 9: Confusion matrix of the improved CNN on the validation set

A plausible interpretation, not a proven cause, is that rose buds and cup-shaped tulips share colours and rounded outlines, and that at 128 × 128 pixels fine petal detail is partly lost; colour variation and cluttered backgrounds may also distract the model. Figure 10 shows the first twelve misclassified images; several are ambiguous, for example images with several species or a small flower in a large background.

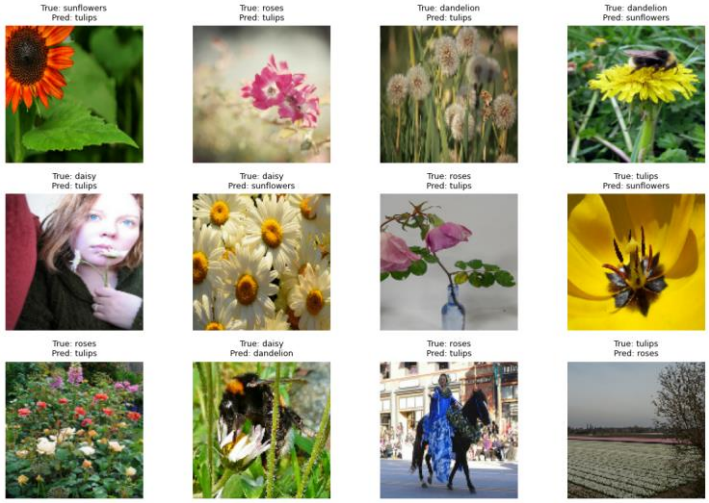


Figure 10: First twelve misclassified validation images (true vs predicted label)

7.4 Example Prediction

The notebook also classifies a sunflower photograph taken from the TensorFlow example images, not from the validation set. The selected model gave 99.3% probability to sunflowers

and 0.7% to tulips (0.0% to the other classes), as shown in Figure 11. This is one illustrative example and not a measure of overall accuracy, which is the 75.34% validation figure.



Figure 11: Prediction on a single new sunflower image

8: Limitations and Future Scope

8.1 Limitations of the Completed Work

A validation accuracy of 75.34% is moderate: about one in four validation images is misclassified. The notebook has no independent test set, and the final model was chosen using the same validation images used to report its performance, so the figure may be slightly optimistic. Results come from a single run, so run-to-run variation is unknown. About 3,670 images in five classes is a small, not very diverse dataset, its curated photographs may differ from user photos taken with other cameras, lighting and backgrounds, and the model cannot reject an image that is not one of the five flowers.

8.2 Future Scope (Proposed, Not Implemented)

The following are proposals only and were not carried out in this project:

- evaluate the final model on a separate, untouched test set to obtain an unbiased estimate;
- collect more diverse images, particularly extra rose and tulip photographs;
- tune hyperparameters (learning rate, dropout rates, filter counts, image size) and test other augmentations;
- compare the current CNN with pre-trained transfer-learning models; and
- build a simple interface for uploading a photograph and viewing the predicted species.

9: Conclusion

This project addressed recognising five flower species from photographs using CNNs built with TensorFlow and Keras. A baseline CNN and an improved CNN with data augmentation and dropout were trained on 2,936 images and evaluated on 734 validation images. The baseline reached 99.18% training accuracy but only 66.89% final validation accuracy, with validation loss rising to 2.2104, which demonstrates overfitting. The improved model had

lower training accuracy (80.45%) but higher validation accuracy (75.34% final, 76.02% best), a validation loss of 0.6872 and a gap of only 5.11 points, so it was selected as the final model. Its macro F1-score is 0.74, with strong results for sunflowers and dandelions and weaker ones for roses, of which 43 of 120 were predicted as tulips. The work shows that regularization can improve generalization even while lowering training accuracy, and that per-class metrics and the confusion matrix expose weaknesses a single accuracy figure hides. Because the results are validation metrics from one run on a small dataset, the model is a learning prototype and not a production-ready system; an independent test set, more data and transfer learning are the natural next steps.

References

[1] TensorFlow Team. (2019). Flower photos dataset [Data set]. Google Cloud Storage. https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz

[2] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16), pp. 265–283. https://www.tensorflow.org/

[3] Chollet, F., et al. (2015). Keras [Software documentation]. https://keras.io/

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[5] Scikit-learn developers. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

[6] TensorFlow Authors. Image classification tutorial. https://www.tensorflow.org/tutorials/images/classification

[7] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

[8] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25, 1097–1105.

[9] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/

[10] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). arXiv:1412.6980.

[11] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(56), 1929–1958.